

Distributed RL Training Systems

pig7selene

September 5, 2026

Abstract

Large-scale language-model reinforcement learning is a distributed dataflow in which a recent policy generates trajectories, reward procedures evaluate them, and a learner turns the resulting token-level records into a new policy version. This chapter explains how Actor, Rollout, Reward, Critic, and Reference roles are placed and coordinated across a cluster; why policy synchronization, bounded rollout staleness, and typed trajectory interfaces are correctness conditions; and how FSDP-backed training, high-throughput inference, queues, recovery, and observability interact. Ray and veRL provide concrete systems examples, but the chapter develops the underlying organization independently of one framework.

1 Introduction

Chapters 16, 18, 19, and 21 define the algorithmic loop of online post-training: sample prompts, generate Rollouts from a behavior policy, evaluate them, construct an update signal, optimize for a bounded number of steps, and refresh the policy. On one accelerator, this can appear to be a sequence of ordinary model calls. At useful scale, however, the calls have incompatible execution profiles. Generation is an autoregressive inference workload whose cost grows with response length and benefits from continuous batching; a policy update is a synchronized training workload with activations, gradients, and optimizer state; reward evaluation may be a model forward pass, a CPU-bound verifier, or a sandboxed program execution. The system must coordinate all three without silently changing the on-policy data contract.

This chapter studies that coordination. It does not re-derive PPO, GAE, GRPO, DAPO, verifier design, or preference modeling. Instead, it treats those procedures as consumers and producers of explicitly versioned records. A **worker** here is a scheduled resource-owning process or group of processes. It is distinct from the **actor** in actor-critic terminology: an **Actor Worker** owns the trainable policy computation, whereas a Ray actor is one programming-model abstraction that can host a worker role.

2 System Roles and the RL Dataflow

An online RL iteration has two kinds of model state. The **behavior policy** π_{θ_v} at version v generates a trajectory. The **learner policy** π_{θ} is the state currently receiving updates. For a bounded part of an iteration they may denote the same checkpoint, but a distributed system must not assume that every worker observes a weight update at the same instant. Chapters 16 and 21 establish why old-policy log probabilities and behavior-policy identity remain part of the PPO- or GRPO-style objective.

Let one transmitted trajectory record be

$$\tau = (x, y, v, \ell_{\text{old}}, m, R, Z, h), \quad (1)$$

where x is a prompt, y is the generated token sequence, v is the behavior-policy version, ℓ_{old} stores behavior-policy token log probabilities, m is the action mask, R is a reward or verifier result, Z denotes optional value predictions or other algorithm-specific fields, and h carries the serialization, sampling, and provenance metadata. The learner may add advantages, returns, reference log probabilities, and losses, but it must not lose the fields that identify the distribution from which y was drawn.

Role	Primary computation	State and failure boundary
Actor or Policy Worker	Policy log probabilities, policy loss, backward pass, and optimizer update.	Trainable policy, optimizer and scheduler state, distributed-training topology, and a checkpoint commit boundary.
Rollout Worker	Autoregressive generation under one published behavior-policy version.	Inference weights, KV Cache, sampling configuration, request identity, and generated-token provenance.
Reward or Verifier Worker	Reward Model scoring, deterministic checking, program execution, or reward composition.	Reward or checker revision, parsing and timeout policy, environment image, and raw score trace.
Critic Worker, when used	Value predictions and value loss for actor-critic algorithms.	Value parameters, optimizer state, target convention, and alignment with the behavior-policy rollout.
Reference Worker, when used	Reference-policy log probabilities or KL-related quantities.	Frozen reference revision, tokenizer and Chat Template compatibility, and response-token masking.

Table 1: A distributed RL system separates roles whose state changes at different rates and whose execution profiles differ. GRPO-like methods may omit a Critic Worker; algorithms without a reference constraint may omit a Reference Worker.

Table 1 is a logical decomposition, not a requirement that every role have a dedicated process. A small system can co-locate several roles, while a large system can partition one role across many devices. The useful invariant is ownership: an observation must retain the exact policy, reward, and formatting identities that produced it even if the physical workers are replaced or moved.

2.1 End-to-End Lifecycle

One conservative synchronous iteration is

$$\begin{aligned} &\text{publish } \pi_{\theta_v} \rightarrow \text{generate } \tau \rightarrow \text{score and validate} \rightarrow \text{construct learning fields} \\ &\rightarrow \text{update } \pi_{\theta} \rightarrow \text{commit } \pi_{\theta_{v+1}} \rightarrow \text{synchronize or serve the new version} \end{aligned}$$

The word **publish** means more than exposing a file path. Rollout Workers need an atomic association between the weights they load and the version identifier they place in Equation 1. The learner must similarly know which behavior version supplied a batch before it evaluates an importance ratio. A system can overlap phases, but only by making the overlap explicit: a trajectory produced by v may be processed while the learner prepares $v + 1$, subject to a declared maximum lag.

This lifecycle also separates **training** from **inference** inside one job. The Actor Worker executes a training forward and backward pass, and may be sharded with FSDP. The Rollout Worker executes Prefill and Decode, maintains a KV Cache, and benefits from the scheduling techniques of Chapters 23–29. Treating its memory and batch policy as though it were a training worker often produces poor utilization or an incorrect view of the job’s capacity.

3 Placement, Colocation, and Scheduling

The first deployment choice is whether roles share accelerators. In a **colocated** design, a training and generation role use the same resource pool at different times. It can avoid copying a large policy between distinct pools and can be economical on a small cluster. Its main cost is phase interference: a training step and a latency-sensitive Decode workload cannot both own the same accelerator memory at full capacity. Switching a model between a sharded training layout and an inference layout can also require reshaping or transferring state.

In a **disaggregated** design, a learner pool, a rollout-serving pool, and optional reward or verifier pools receive separate resources. This permits generation to continue while the learner updates, and it allows each pool to use the parallelism appropriate to its workload. The price is extra weight synchronization, data movement, and coordination. There is no universally better deployment. The decision depends on model size, rollout-to-update ratio, sequence lengths, verifier cost, network bandwidth, and the degree of staleness that the algorithm permits.

Placement choice	What it can improve	What must be measured
Colocate policy training and Rollouts	Avoided duplicate model residency and possibly cheaper local handoff.	Phase switching, allocator peaks, interference, and idle time between phases.
Disaggregate learner and Rollouts	Independent training and serving schedules; role-specific parallelism.	Weight-transfer cost, rollout lag, network use, and trajectory-queue growth.
Separate Reward or Verifier pool	Scales expensive model scoring or heterogeneous checking independently.	Queue delay, sandbox capacity, reward-version consistency, and result ordering.
Co-locate tightly coupled ranks	Locality for high-bandwidth collectives or model-parallel groups.	Node capacity, fault domain, and whether packing prevents other work from scheduling.

Table 2: Placement is a workload-specific resource decision. A lower weight-transfer cost can be outweighed by reduced overlap, while a more elastic layout can be limited by version lag or the network.

Ray is one useful orchestration layer because it expresses both persistent actor-style computation and short-lived tasks through a distributed runtime [1]. Its placement groups reserve resource bundles atomically and let a job pack related resources for locality or spread them across nodes for a different fault domain [2]. These mechanisms are useful for assigning a Tensor-Parallel rollout group, an FSDP learner group, and a CPU-based verifier service, but they do not choose an RL algorithm or make an unsafe placement correct. In particular, every role should declare its CPU, GPU, memory, and any topology-sensitive requirements rather than relying on scheduler defaults.

3.1 GPU Allocation and Backpressure

The slowest stage determines steady-state throughput. If Rollout Workers generate completions faster than reward evaluation or training can consume them, the trajectory queue grows until host memory, object-store capacity, or durable storage becomes the bottleneck. If the learner is faster than generation, it waits for fresh on-policy examples or is tempted to reuse stale trajectories beyond its declared update regime. The appropriate response is **backpressure**: bounded queue capacity and admission rules that slow an upstream producer, limit prompts in flight, or defer lower-priority work before an unbounded buffer silently changes the job’s failure mode.

For an average rollout arrival rate λ_{roll} and downstream consumption rate μ_{down} , a queue is stable only in the simplified steady-state sense that

$$\lambda_{\text{roll}} < \mu_{\text{down}}. \quad (2)$$

The inequality is not a complete performance model. Arrival times are bursty, response lengths vary, and a single slow verifier can delay a group. Its value is diagnostic: adding Rollout GPUs while μ_{down} is fixed can raise the queue length rather than improve completed RL iterations. Queue occupancy, waiting time, rejection rate, and the policy-version distribution of queued trajectories must therefore be monitored together.

4 Synchronization, FSDP, and Inference Engines

Policy synchronization is the boundary between learning and generation. After an update, a colocated system may reuse resident parameters after an explicit phase transition. A disaggregated system must publish a consistent new weight version to its Rollout Workers. The mechanism can range from a checkpoint-like artifact to an in-memory transfer or a backend-specific resharding path. What matters algorithmically is that no rollout record claims version $v + 1$ unless the generator actually used parameters associated with $v + 1$.

Define the learner’s accepted version lag for trajectory τ as

$$\Delta_{v(\tau)} = v_{\text{learner}} - v_{\text{behavior}(\tau)}, \quad 0 \leq \Delta_{v(\tau)} \leq \Delta_{\text{max}}. \quad (3)$$

The upper bound Δ_{max} is a data-contract choice, not a cosmetic metric. PPO and GRPO use behavior-policy log probabilities in their ratios, so a large lag increases distribution mismatch and makes a nominally on-policy update less reliable. Chapter 21 treats this as an algorithmic condition; the system

realizes it with atomic version publication, queue admission or eviction, and an explicit decision about unfinished requests during a refresh.

FSDP is relevant on the learner side because actor and critic models may require parameter, gradient, and optimizer-state sharding. Chapter 12 explains the All-Gather and Reduce-Scatter lifecycle, memory peaks, and checkpoint representations for that role. It should not be assumed that an FSDP live layout is directly consumable by an inference engine. The synchronization interface has to map the logical model state to the rollout engine’s accepted partitioning and precision. A fast transfer that changes tied weights, tokenizer assumptions, parameter names, or model revision identity is not a valid optimization.

High-throughput engines such as vLLM are relevant to Rollout Workers because they provide efficient autoregressive generation and KV Cache management; the vLLM system design is described by Kwon et al. [3]. They do not make rollouts an ordinary serving request. An RL system must additionally retain sampled token IDs, old-policy log probabilities, stop reasons, action masks, and the exact sampling configuration. veRL’s HybridFlow design is a representative example of separating the RL dataflow from the mapping of training and generation computation to devices, and it explicitly integrates FSDP- and inference-engine backends [4], [5].

5 Trajectory Data Plane and DataProto

The trajectory queue is an interface between independently scheduled roles, not an incidental Python list. It must carry fixed-shape or padded tensor fields alongside variable-length and structured metadata. A useful schema partitions the record into three classes:

- tensor fields, such as token IDs, attention and action masks, old log probabilities, values, rewards, returns, and advantages;
- per-sample non-tensor fields, such as prompt identifiers, tool traces, verifier diagnostics, stop reasons, and source records; and
- batch metadata, such as behavior-policy version, tokenizer and Chat Template identity, reward and reference revisions, padding convention, and the producer’s serialization policy.

The separation prevents two common mistakes. First, a field that is indexed with each response cannot be treated as one batch-global value; prompt and response metadata must remain aligned when a group is repeated, filtered, packed, or shuffled. Second, a tensor whose dtype, shape, or mask changes must not be smuggled into opaque metadata, where a learner cannot validate it before computing a loss. veRL’s DataProto is one concrete interface of this kind: it separates batched tensors, non-tensor batch data, and metadata and enforces a shared leading batch dimension for its declared fields [5].

DataProto is not a new RL objective. It is a data-interface discipline that lets a controller or worker group dispatch, slice, repeat, and merge records without guessing which fields are per-token, per-sample, or per-batch. A framework-independent system needs the same distinctions even if it uses a different container. The stable identity of Equation 1, including group membership for GRPO and selection history for DAPO-like filtering, remains more important than the container’s class name.

5.1 Queues, Storage, and On-Policy Lifetime

On-policy trajectories have a short statistical lifetime. Once their policy version exceeds the accepted lag in Equation 3, they should be rejected, explicitly reweighted by a declared algorithm, or retained only for diagnostics. An online RL queue is therefore not automatically an off-policy replay buffer. Keeping old trajectories because they are expensive to generate can improve hardware utilization while degrading the approximation that justified the update.

The system must specify whether a queue is memory-resident, object-store-backed, or durably persisted. Memory-resident transfer minimizes latency but makes worker failure and capacity pressure important. Durable storage improves recovery and auditability but adds I/O and serialization cost. In either case, each item needs a unique identifier, producer version, completion state, checksum or

integrity boundary where appropriate, and idempotent consumption semantics. A retry must not train twice on the same accepted trajectory merely because an acknowledgement was lost.

6 veRL as a Concrete Systems Model

veRL is an open-source realization of the HybridFlow design. HybridFlow observes that an RLHF dataflow has large distributed training or generation programs at its nodes and many-to-many state transfers at its edges; it therefore combines centralized orchestration with distributed execution inside worker groups [4]. This is a useful architectural lesson: a single Python-level training loop should not have to micromanage every collective inside FSDP or every model-parallel inference rank.

In veRL terminology, a controller coordinates role-specific workers and their data movement, while model backends can supply the training and rollout implementations. Its public documentation emphasizes flexible mapping of models onto different GPU sets and integrations with FSDP, Megatron-LM, and vLLM [5]. DataProto gives those components a common batch interface. These names should be read as one concrete implementation, not a universal taxonomy: another system can use service endpoints, message queues, or a compiled runtime and still need the same policy-version, trajectory-schema, placement, and recovery contracts.

The framework example also exposes a limitation. A centralized controller that receives every full trajectory can become a data-plane bottleneck even if GPU workers are fast. Recent veRL documentation identifies this explicitly for controller-routed DataProto transfer and motivates more streamed dispatch paths [5]. The general conclusion is not that every system should be fully asynchronous. It is that control-plane decisions, bulk trajectory transport, and distributed model computation should be profiled separately; otherwise a faster rollout engine can simply move the bottleneck to orchestration.

7 Recovery, Observability, and Debugging

Fault recovery begins by deciding which lifecycle boundary is committed. A learner checkpoint should include the policy, critic where applicable, optimizer and scheduler states, random states, prompt-data cursor, and FSDP or other distributed-state metadata described in Chapters 11 and 12. The RL-specific addition is a record of the currently published behavior version, reference and reward identities, queue or rollout-buffer state, accepted-lag rule, sampling configuration, and any curriculum or selection state. A restored learner that cannot identify which trajectories are valid for its next update has not restored the same RL job.

Worker failures need role-specific handling. A failed Rollout Worker can often retry an unfinished request if the published policy version and sampling seed or request identity are preserved. A verifier retry must retain its checker image, timeout, and external environment version. A failed learner during an update requires rollback to a checkpointed commit boundary; partially applied optimizer updates or partially acknowledged trajectory consumption should not be silently combined with a retry. Ray actors can be configured with restart and task-retry policies, but defaults and placement-group behavior must be made explicit rather than assumed [2].

Observed signal	First question	Likely system boundary
Growing rollout queue	Which stage has lower completed-item throughput: generation, reward, or learner consumption?	Backpressure, reward capacity, learner step time, or stale-data admission.
High GPU use but low RL iterations per hour	Are inference and training phases contending, or is queue / serialization time exposed?	Placement, weight synchronization, host transfer, or controller bottleneck.
Rising policy lag	Which weight version generated each queued group, and is the learner accepting it?	Publication cadence, rollout duration, queue backlog, or missing admission control.
Reward result mismatch	Did policy, reward, parser, and Chat Template versions agree for the same token sequence?	Data schema, reward worker, response boundary, or stale service.
Collective stall or learner OOM	At which FSDP lifecycle boundary did the failure first occur?	Wrapping, prefetch, checkpointing, rank divergence, or overlap policy.

Table 3: Distributed RL diagnostics should locate a failure at a dataflow boundary before changing an algorithmic hyperparameter. A policy-loss anomaly can originate in a queue, version, or reward contract rather than in PPO or GRPO itself.

The primary system metrics are not interchangeable with learning metrics. Log requests and output tokens per second, queue occupancy and age, reward latency, learner valid tokens per second, weight-synchronization duration, GPU memory and utilization by role, and FSDP collective wait. Alongside them, retain the algorithmic signals of Chapters 16, 18, and 21: reward distribution, advantage statistics, KL, entropy, ratio and clip fractions, pass rate, response length, selection rate, and policy lag. A rising reward with an increasing backlog does not establish improvement; it may only show that the system has changed which examples reach the learner.

8 Implementation Contracts

The role contract must declare which worker groups own policy, rollout, reward or verifier, critic, and reference computations; the model revision and tokenizer / Chat Template used by each; the resource bundle and placement policy; and whether roles are colocated or disaggregated. It must separately state the training and inference parallelism plans, including FSDP ownership, rollout-engine partitioning, precision, and the version-publication mechanism. No worker may claim a policy version without an atomic association to the parameters it used.

The trajectory contract must define the schema in Equation 1, tensor shapes and dtypes, padding and action-mask semantics, per-sample versus batch metadata, serialization format, queue capacity, producer acknowledgement, deduplication key, and maximum accepted version lag. It must preserve group membership, raw reward components, checker traces, and selection decisions where an algorithm needs them. Tests should deliberately repeat, filter, truncate, and reorder a batch to verify that all aligned fields follow the same sample indices.

The recovery and observability contract must define publish, enqueue, consume, update, and checkpoint commit boundaries; retry and idempotency rules; durable-artifact locations; and the conditions under which a queued trajectory is discarded after a restart. It must log role-specific throughput, queue age, policy lag, synchronization time, reward latency, memory peaks, collective waits, and all algorithmic diagnostics required by the chosen objective. An end-to-end test should kill a worker at each boundary, restore from a fresh process, and show that the next learner update consumes a well-defined set of trajectories.

9 Summary

Distributed RL training for LLMs is a versioned dataflow, not merely a larger PPO or GRPO batch. Actor Workers optimize a sharded training state; Rollout Workers run an autoregressive inference workload; Reward, Verifier, Critic, and Reference Workers provide different forms of evaluation or

control. Correctness depends on preserving the behavior-policy version, sampled log probabilities, token masks, reward provenance, and formatting identity alongside every trajectory.

Placement and scheduling determine whether these roles interfere or overlap. Colocation can avoid weight movement but forces phase sharing; disaggregation enables role-specific scaling but creates synchronization, queue, and staleness costs. FSDP handles learner-state capacity, while inference engines such as vLLM address high-throughput generation. Ray and veRL illustrate how resource bundles, worker groups, controller logic, and typed data containers can realize the resulting system, but no framework removes the need for bounded backpressure, explicit policy publication, durable recovery boundaries, and end-to-end observability.

References

- [1] P. Moritz *et al.*, “Ray: A Distributed Framework for Emerging AI Applications,” in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, USENIX Association, 2018, pp. 561–577. [Online]. Available: <https://www.usenix.org/conference/osdi18/presentation/moritz>
- [2] Ray Contributors, “Placement Groups.” 2026. [Online]. Available: <https://docs.ray.io/en/latest/ray-core/scheduling/placement-group.html>
- [3] W. Kwon *et al.*, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *Proceedings of the 29th Symposium on Operating Systems Principles*, Association for Computing Machinery, 2023, pp. 611–626. doi: 10.1145/3600006.3613165.
- [4] G. Sheng *et al.*, “HybridFlow: A Flexible and Efficient RLHF Framework,” *arXiv preprint arXiv:2409.19256*, 2024, doi: 10.48550/arXiv.2409.19256.
- [5] verl Contributors, “verl: Volcano Engine Reinforcement Learning for LLMs.” 2026. [Online]. Available: <https://github.com/verl-project/verl>